Rúbrica juego RPG

Techos de nota

- No compila: máximo 3.00.
- Usa estructuras explícitamente prohibidas: máximo 3.00.
- No supera la defensa: máximo 3.00.

Un techo no asigna directamente esa calificación: si se aplica un máximo de 3 puntos, la práctica pasa a corregirse de 0 a 3.

Penalizaciones globales

- UML no corresponde con el código: -2.00.
- Código comentado, muerto o de pruebas: -1.00.
- Nombres o formato muy deficientes: -0.50.
- Más de un return en una función: -1.00. Pendiente decidir alcance.
- Break fuera de switch: -1.00. Pendiente decidir alcance.
- Continue: -1.00. Pendiente decidir alcance.

UML - 3 puntos

Relaciones - 0.5

- Falta una relación o su tipo es incorrecto: -0.15 por caso.
- Faltan multiplicidades: -0.10 por caso.

Clases - 0.5

- Falta una clase relevante: -0.20 por caso.
- Aparece una clase que no corresponde: -0.10.
- Tipo de elemento UML incorrecto: -0.20.

Contenido - 2

- Faltan atributos: -0.25.
- Faltan métodos principales: -0.25.
- No se indican visibilidades: -0.25.
- Faltan tipos, parámetros o retornos: -0.25.
- No se reflejan excepciones propias: -0.20.

Excepciones - 1 punto

- Faltan excepciones propias necesarias: -0.25 por cada una.
- Excepción no lanzada en sitio adecuado: -0.10 por caso.
- Excepción no capturada en sitio adecuado: -0.10 por caso.
- Captura genérica que oculta errores o vacía el flujo: -0.10.
- No gestiona bien entrada/salida: -0.20.

Main - 1 punto

Se valora por grado de cumplimiento: 0, 0.25, 0.5 o 1 sobre su valor máximo.

- Creación y utilización correcta de objetos.
- Lectura/carga inicial de ficheros integrada correctamente en el arranque, aunque esté delegada en funciones auxiliares o en otra clase.
- Separación del `main` en funciones auxiliares cuando corresponde, evitando un main monolítico.
- Ausencia de lógica de dominio en `main`.

Implementación general - 5 puntos

Cada subapartado se valora por grado de cumplimiento: 0, 0.25, 0.5 o 1 sobre su valor máximo.

Navegación por menús - 0.70

- Clase `Menu` bien modularizada en métodos auxiliares.
- Delegación correcta en las clases del sistema, respetando la encapsulación.
- Flujo de uso comprensible para el usuario.
- Lógica de navegación sencilla y eficiente, sin duplicar código innecesariamente.

Equipar y desequipar - 1.20

- Movimiento correcto entre inventario y equipo.
- Reemplazo de equipo sin perder ni duplicar objetos.
- Actualización coherente de ataque y defensa.
- Uso de `IEquipable` y polimorfismo.
- Distinción correcta entre arma y armaduras.
- Encapsulación entre personaje, inventario y equipo.

Comprar en tienda - 1.00

- Movimiento correcto del objeto desde la tienda al inventario del personaje.
- Actualización coherente del dinero del personaje y del precio del objeto.
- La tienda deja de ofrecer el objeto comprado.
- Conservación de la información completa del objeto comprado, incluido metal.
- Encapsulación entre menú, tienda, personaje e inventario.

Guardar personaje - 0.70

- Escritura del estado actual del personaje en fichero.
- Conservación de datos principales, equipo e inventario.
- Formato coherente con el formato de lectura.
- Uso adecuado de escritura de ficheros sin mezclar responsabilidades.
- Encapsulación: el guardado no debe requerir manipular desde fuera las estructuras internas del personaje.

Uso de `toString` - 0.20

- Sobrescritura correcta de `toString` en las clases donde aporta información.
- Salida legible y útil para revisar el estado del sistema.
- Utilización de `toString` de forma adecuada en las clases.

Herencia de `Objeto` - 1.10

- `Objeto` como clase base.
- Herencia correcta en arma y armadura.
- Atributos y comportamiento común ubicados en la clase base.
- Constructores bien encadenados.
- Ausencia de duplicación innecesaria entre clases hijas.

Uso de `enum` - 0.10

- Uso cuando hay conjunto cerrado de valores.
- Integración real en la lógica.